

Informe Técnico Exhaustivo: Simulador de Carreras OpenGL 3D

Este documento detalla todas las especificaciones, arquitectura de software, mecánicas matemáticas de física y características visuales del proyecto **Simulador de Carreras**. El proyecto ha sido programado desde cero en C++ utilizando la API gráfica nativa OpenGL (3.3+ Core Profile) garantizando un rendimiento óptimo de 60+ FPS y control total sobre el pipeline gráfico.

1. Visión General y Dependencias

El simulador es un videojuego de carreras en formato contrarreloj y circuito cerrado. El jugador compite contra el cronómetro para completar 2 vueltas con un coche de características arcade, físicas de choque y deslizamiento personalizadas, en un entorno con ciclos de luz dinámicos y un HUD incrustado en el propio contexto de vídeo.

Dependencias principales y Librerías:

- **OpenGL & GLAD:** Acceso y enrutamiento a funciones nativas de la tarjeta de vídeo.
- **GLFW:** Creación de contextos de ventana y gestión eventos de *hardware* (teclado).
- **GLM (OpenGL Mathematics):** Librería `header-only` para trabajar de manera eficiente con `vec3`, `mat4`, rotaciones matriciales y proyecciones.
- **stb_image:** Lectura de mapas de bits (.png, .jpg) para aplicar como texturas en la RAM de video.
- **Dear ImGui:** Librería gráfica para generar Interfaces de Usuario (HUDs) instantáneos de bajísimo coste computacional superpuestos en la ventana 3D.

2. Arquitectura de Software y Archivos Base

El proyecto se construye de manera modular para separar la geometría, la lógica de simulación y el código de los *Shaders* de la GPU.

2.1 `main.cpp`

Es el corazón del proyecto. Contiene:

- **Bucle de Juego (Game Loop):** Implementa un bucle dinámico *DeltaTime* (`dt`), asegurando que las velocidades y cálculos de rotación se mantengan consistentes independientemente de los fotogramas por segundo a los que corra la máquina.
- **Manejo de Estados:** Instancias globales de la estructura `Coche` y variables de estado del mundo (`esDeNoche`, `tiempoCarrera`).
- **Renderizado Múltiple:** Rutinas iterativas (`dibujarEntorno`, `dibujarCoche`) que instancian geometría base escalando, trasladando y rotando matrices antes de mandarlas al shader.

2.2 Geometría y `geometria_coche.h`

En vez de cargar complicados modelos 3D (`.obj`), todo el mundo está formado por primitivas geométricas construidas vértice a vértice, promoviendo la eficiencia.

- Define `baseVerticesCubo`: *Array* estático que contiene Posición (`x, y, z`), Normales (`nx, ny, nz`) para que la luz rebote correctamente, y Coordenadas UV (`u, v`) para texturas.
- **Generación Procedural (`generarCilindro`):** La estructura del neumático no está prescrita, sino que se dibuja matemáticamente mediante una función de triangulación utilizando trigonometría

(`sin` , `cos`) dividiendo un círculo perfecto de radio 0.5 en 24 segmentos, permitiendo ruedas verdaderamente circulares.

2.3 Shaders (Pipeline Programable)

- **shader.vert (Vertex Shader):** Transforma coordenadas espaciales desde "Local" a "Mundo", a "Vista", y finalmente a "Proyección" (usando frustum de perspectiva). Además, procesa las normales usando `mat3(transpose(inverse(modelo)))` para evitar deformaciones indeseadas.
- **shader.frag (Fragment Shader):** Recibe las coordenadas por píxel y calcula:
 - Textura base combinada con opacidad `alphaObj`.
 - **Iluminación Phong:** Modelo matemático para reflexiones Difusas y Especulares, operando focos (SpotLights) limitados por ángulos internos (15°) y externos (25°), atenuando la luz de manera inversa al cuadrado de la distancia.
 - **Materiales Emisivos (esEmissivo):** Ignora el modelo de sombras para piezas que emiten su propia luz (como el fuego de propulsión).

3. Físicas de Simulación y Controles

Las mecánicas intentan emular un vehículo de inercia acentuada. Todas actúan en el método `actualizarFisicas()`.

Cinemática

- **Dirección (`A` , `D`):** El giro se acumula en un radio virtual, topado a -40° / +40°. Implementa un sistema de "Auto-retorno" muy agresivo que endereza el volante rápidamente al soltar la tecla.
- **Caja de Cambios:** Consta de 6 marchas con velocidad límite de 10 km/h en la 1° hasta 60 km/h en la 6°.
 - Las marchas pueden ser subidas manualmente (`TecLas 1 a 6`).
 - Al frenar o perder velocidad drásticamente, el sistema efectúa *Downshifts* (reducciones de marcha automáticas) buscando la marcha óptima inferior a la velocidad actual.
- **Tracción sobre Asfalto vs. Hierba:** Si el radio polar del coche sale del intervalo asfáltico (radios 80 a 120), el juego reduce inmediatamente la velocidad punta permitida a **8 km/h**, actuando como factor de pérdida de tracción extrema.

Mecánica Arcade: Sistema de Nitro (`N`)

Al pulsar la letra `N`, se sobrescriben momentáneamente las físicas:

- **Aceleración Inicial:** Otorga un *Boost* multiplicativo de la velocidad actual del +50%.
- **Bypass de Límites:** Mientras dura la sobrecarga de inyección (1.5 segundos), el límite de marcha salta a 100 km/h y, más significativamente, anula la tracción baja de la hierba, permitiendo cruzar campo a través a un máximo de **20 km/h**.
- Está regulado por un temporizador interno que bloquea reutilizar la propulsión por **15 segundos** (*Cooldown*).

4. Diseño del Escenario e Inteligencia del Renderizado

La pista no es un modelo plano. Se calcula algorítmicamente en `dibujarEntorno()`.

- **Generación de Terreno Instanciado:** La base de datos no guarda miles de losas de terreno. El bucle cruza el plano X/Z y por cada sección:

1. Utiliza `getDistanciaOval()` la cual distorsiona matemáticamente un radio elíptico en X y añade $+ \sin(x * 0.05f) * 20.0f$ a Z. Esto da como resultado curvas y chicanes complejas puramente creadas por funciones senoidales continuas.
 2. Decide qué material pintar (Asfalto, Hierba, o Línea de Meta a cuadros) basándose de nuevo en el radio polar exacto de dicha losa de renderizado.
- **Muros Dinámicos:** Muros protectores que envuelven el límite exterior y la isleta interior (creando una escapatoria física perimetral visible de seguridad en caso de derrape largo).

5. Diseño de Interfaces (HUD y Feedback de Consola)

- **Integración Dear ImGui:** Se han sobrepuesto paneles informativos con estilo transparente oscuro renderizados sobre la imagen en tiempo real antes de intercambiar el búfer. Se gestiona un HUD lateral en pantalla.
- **Barra de Progreso Funcional:** Para el uso táctico del Nitro, la UI cuenta con una `ImGui::ProgressBar` que lee el valor interno `nitroCooldown` referenciado a su estado inicial para proveer relleno porcentual fluido.
- **Terminal Sincronizada:** Entendiendo que la consola de Linux puede ser usada de registro de depuración, se incluyó un *logger* asíncrono que imprime métricas exactas dos veces por segundo superponiéndose en línea de comando usando Retorno de Carro `\r`, evitando desbordar la RAM al no imprimir 60 líneas por segundo.

6. Iluminación y Texturización de Contextos

- **SkyDome Dinámico:** Un cubo no, una esfera con texturizado interno de `1080` triángulos (`vaoEsfera`), escalada gigante (`scale 100.0f`) a la cual se le anula temporalmente la escritura del *Z-Buffer* (`glDepthMask(GL_FALSE)`). Se traslada a la misma posición que la cámara fotograma a fotograma creando una ilusión óptica de ser "El Cielo" a distancia infinita.
- **Efectos de Noche (T):** Se atenúa en el Shader la Luz Ambiente casi por completo cambiando el material reflectivo y el cielo.
- **Los Faros (F):** Emulan faros parabólicos de Xenon. Posicionan dos orígenes dinámicos frente al morro, enviando los vectores de corte calculados en el fragmento.
- **Fuego de Exhaustación (Emisivo):** Genera parpadeo escalado en base a `sin(glfwGetTime() * 40.0f)`.

7. Cámaras Multidimensionales

Gestionadas mediante rotación polar y transformaciones afines de `glm::lookAt`:

1. **Cámara Libre Persecución (8):** Ángulo de grúa, sigue al coche sumando un vector opuesto a la rotación de chasis.
2. **Primera Persona - Dashboard (7):** Altura hundida al ras del techo con punto de mira frontal anclado.
3. **Cenital / Victoria (9):** Distancia masiva del eje vertical. Pasa de un campo de visión periférico a ser activada forzosamente tras registrar en `cruzandoMetaAhora` que la coordenada Z espacial está cruzando la bandera de cuadros por segunda vez consecutiva validada direccionalmente (`anguloChasis` paralelo a la pista).